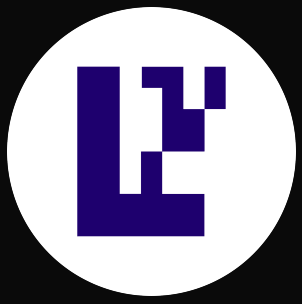
Security Assessment Report



Redistribution Slash Delay Upgrade

May 2026

Prepared for EigenLayer

Table of Contents

certora

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
EigenLayer - Redistribution Slash Delay Upgrade - PR #1753	https://github.com/Layr-Labs/eigenlayer-contracts	Initial commit: 7e02c2e Final commit: f53d878	Solidity / EVM

Project Overview

This document describes the security review of **EigenLayer - Redistribution Slash Delay Upgrade - PR #1753**. The work was undertaken from **May 11th, 2026** to **May 12th, 2026**.

The following contract list is included in our scope:

src/contracts/**/*.sol (changes introduced by PR #1753)

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

EigenLayer is an Ethereum restaking protocol that allows stakers to reuse staked assets to secure external services, called AVSs, through delegated operators and slashable commitments. PR #1753 introduces a 50,400-block slash resolution delay before ERC20 strategy shares marked for burn or redistribution can be cleared and transferred out of StrategyManager. The upgrade also adds a dedicated pause flag for burn/redistribution clearing, giving governance an emergency stop during the resolution window. Separately, it narrows duration vault token restrictions by removing the generic isBlacklisted[token] checks and replacing them with a specific prohibition on EIGEN and bEIGEN duration vaults.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

certora

Threat and Security Overview

Assets

- ERC-20 collateral held in StrategyBase-derived strategies and duration vaults
- Slash-redistribution payouts routed through IStrategy.withdraw
- Pending-slash bookkeeping (_burnOrRedistributableShares, _slashResolutionBlock)
- Storage layout integrity of the StrategyManager proxy

Actors

- Pauser (controls PAUSED_BURNING_AND_REDISTRIBUTION)
- DelegationManager (sole indirect writer of _slashResolutionBlock)
- AllocationManager (issues slashIds, sets redistribution recipient)
- Permissionless clearer (anyone after delay elapses)
- Timelocked upgrade multisig
- Off-chain monitoring

Trust Assumptions

- AllocationManager issues monotonically increasing, never-reused slashIds
- Redistribution recipient is set once and never mutates
- Pauser will not weaponize the all-or-nothing pause flag
- Pre-upgrade pending slashes are intentionally grandfathered (resolutionBlock == 0 → immediately clearable)
- The multiSend upgrade bundle is atomic; partial application is impossible
- EIGEN/bEIGEN immutables are non-zero and will never need to rotate
- Off-chain incident response can act within 7 days

Attack Vectors

- Pre-delay redistribution rug if the resolution-block stamp is missing, zero, or bypassed
- Permanent freeze of all redistribution via the all-or-nothing pause flag (no per-opSet granularity)
- Storage corruption from a botched gap reduction during upgrade
- OOG / DoS on batch clearBurnOrRedistributableShares due to unbounded .keys() copy
- Bypass of the EIGEN/bEIGEN ban via deployNewStrategy (no constructor-immutable check there)
- Removed runtime blacklist on duration-vault deposits — blacklisting a token no longer freezes deposits into existing vaults
- Non-atomic upgrade hazard: if the multiSend bundle is ever split, mismatched StrategyFactory + DurationVaultStrategy impls coexist
- Future internal caller of _clearBurnOrRedistributableShares bypassing pause + delay (gating lives only on the external entrypoints)

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	5	5	3
Total	5	5	3

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

certora

Detailed Findings

ID	Title	Severity	Status
I-01	Duration vaults have no proactive token-level denylist	Informational	Acknowledged
I-02	Stale documentation for duration-vault blacklist behavior	Informational	Fixed
I-03	Slash resolution delay can temporarily preserve TVL cap usage for slashed assets	Informational	Acknowledged
I-04	Legacy burnShares() is not covered by the new burn/redistribution pause	Informational	Fixed
I-05	BurnOrRedistributableSharesIncreased event does not emit the slash resolution block	Informational	Fixed

Informational Severity Issues

I-01: Duration vaults have no proactive token-level denylist

Files:

src/contracts/strategies/DurationVaultStrategy.sol
src/contracts/strategies/StrategyFactory.sol

Description:

PR #1753 removes the generic `isBlacklisted[token]` checks from duration vault deployment and deposits, replacing them only with an EIGEN/bEIGEN prohibition. As a result, any user can permissionlessly deploy a duration vault for any other token, including tokens blacklisted for regular strategies, and the factory will automatically whitelist the new vault in `StrategyManager.strategyIsWhitelistedForDeposit`. This also makes the vault eligible to be referenced in flows that use the deposit whitelist as a strategy allowlist, such as `RewardsCoordinator._validateCommonRewardsSubmission()`.

This may be intended for pre-existing LSTs like stETH/cbETH, where the regular blacklist is used to prevent duplicate canonical strategies. However, after this change there is no proactive token-level mechanism to block unwanted or unsafe tokens specifically for duration vaults. Governance can only reactively remove individual vault strategies after deployment. Even if the current blacklist is mainly a duplicate-strategy guard, future token-risk or operational needs may require blocking a token from creating strategies, and duration vaults would currently bypass that control.

Recommendations:

Consider introducing a separate duration-vault-specific token denylist, distinct from the regular strategy blacklist, and check it both in `deployDurationVaultStrategy()` and `DurationVaultStrategy.beforeAddShares()`. This would preserve support for intended blacklisted LSTs while retaining a proactive control for tokens that should not be used in duration vaults.

Customer Response:

Acknowledged. It is intended design to allow all tokens.

I-02: Stale documentation for duration-vault blacklist behavior

Files:

docs/core/DurationVaultStrategy.md

Description:

PR #1753 removes the generic `isBlacklisted[token]` checks from duration vault deployment and deposits, leaving only the EIGEN/bEIGEN prohibition. However, the duration vault documentation still says that the token must not be blacklisted during deployment, that `beforeAddShares()` rejects blacklisted underlying tokens, and still lists the removed `UnderlyingTokenBlacklisted` error.

Recommendations:

Update the documentation to state that duration vaults may be deployed for blacklisted tokens except EIGEN/bEIGEN and remove references to `UnderlyingTokenBlacklisted`.

Customer Response:

Fixed in f53d8783.

Fix Review:

Fix confirmed. The stale blacklist references were removed from the documentation.

I-03: Slash resolution delay can temporarily preserve TVL cap usage for slashed assets

Files:

src/contracts/strategies/StrategyBaseTVLLimits.sol

Description:

PR #1753 introduces a mandatory slash resolution delay before slashed ERC20 strategy shares can be cleared and the underlying tokens can be transferred to the burn or redistribution recipient. For capped strategies, TVL limits are based on the strategy's physical token balance, so slashed assets
continue to count toward the cap while they remain inside the strategy during the delay.

```
function _beforeDeposit(
    IERC20 token,
    uint256 amount
) internal virtual override {
    require(amount <= maxPerDeposit, MaxPerDepositExceedsMax());
    require(_tokenBalance() <= maxTotalDeposits,
BalanceExceedsMaxTotalDeposits());

    super._beforeDeposit(token, amount);
}
```

As a result, a strategy that is at or near its TVL cap may not regain deposit headroom until the delayed clearing transaction is allowed and executed. This was possible before if clearing was delayed operationally, but the PR makes the delay mandatory.

Recommendations:

Document this as an expected consequence of delayed slash settlement for capped strategies.

Customer Response:

Acknowledged. This is expected behavior and the admin has the ability to adjust caps.

I-04: Legacy burnShares() is not covered by the new burn/redistribution pause

Files:

src/contracts/core/StrategyManager.sol

Description:

PR #1753 introduces `PAUSED_BURNING_AND_REDISTRIBUTION` and applies it to the new burn/redistribution clearing functions, but the legacy `burnShares()` path is not gated by this pause flag. `burnShares()` operates on the deprecated `burnableShares` mapping, and we do not see production code currently increasing `StrategyManager.burnableShares`, so this is mainly a consistency and future-proofing concern. However, if any legacy state exists where `burnableShares[strategy]` is non-zero, enabling the new emergency pause would not prevent those shares from being burned through the legacy path.

Recommendations:

For consistency, consider adding `onlyWhenNotPaused(PAUSED_BURNING_AND_REDISTRIBUTION)` to `burnShares()` or explicitly document this behaviour.

Customer Response:

Fixed in f53d8783.

Fix Review:

Fix confirmed. The pause guard was added to the legacy burnShares() path.

I-05: BurnOrRedistributableSharesIncreased event does not emit the slash resolution block

Files:

src/contracts/core/StrategyManager.sol

Description:

PR1753 introduces delayed slash clearing by storing a `_slashResolutionBlock` for each `(operatorSet, slashId)` when burn-or-redistributable shares are first recorded. However, the existing `BurnOrRedistributableSharesIncreased` event only emits the `operatorSet`, `slashId`, `strategy`, and `shares`, without emitting the block after which the slash can be cleared which is an important parameter. Offchain indexers and monitoring that rely on events must either derive the value from transaction block number and the protocol constant, or perform an additional `getSlashResolutionBlock` call before the slash is fully cleared.

Recommendations:

Consider including the slash resolution block in `BurnOrRedistributableSharesIncreased`, or introducing a dedicated slash-level event:

```
event SlashResolutionBlockSet(
    OperatorSet operatorSet,
    uint256 slashId,
    uint32 resolutionBlock
);
```

The event should be emitted only when the resolution block is first initialised for a given `(operatorSet, slashId)`, so when the slash ID is first added to the pending slash set.

Customer Response:

Fixed in f53d8783.

Fix Review:

Fix confirmed. The slash resolution block is now emitted in the introduced SlashResolutionBlockSet event.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.